

NOTES

Introduction to SQL & Relational Databases



Introduction to SQL and Relational Databases

Understanding Databases

What is a Database?

A **database** is a collection of organised data or information that can be easily accessed, managed, and updated. Databases are usually managed through a Database Management System (DBMS), which serves as an interface between the database and its users or applications, allowing for data manipulation and management. Examples include storing product information for an e-commerce website.

Why to use a Database?

1. **Data Organization:** Databases provide a structured way to store large volumes of data, ensuring it is easy to access, update, and manage. For example, an e-commerce platform can store product details, customer information, and orders in separate, organised tables.
2. **Data Integrity:** Databases ensure that data remains accurate and consistent across multiple users and applications. This integrity is maintained through constraints and rules that prevent invalid data entries, which is crucial for reliable reporting and analysis.
3. **Efficient Data Retrieval:** With SQL (Structured Query Language), databases allow for complex queries to retrieve

specific information quickly. For instance, you can easily find all customers who made purchases in the last 30 days.

4. **Data Relationships:** Relational databases can link tables (e.g., connecting customer orders to customer profiles), enabling more complex and interconnected data storage.
5. **Scalability:** Databases are built to handle large amounts of data as your application or business grows. They can scale horizontally or vertically to meet the demands of increasing data and users.
6. **Data Security:** A Database Management System (DBMS) provides robust security features, including user authentication and access controls, safeguarding sensitive information from unauthorised access. This is vital for protecting business data and maintaining compliance with regulations.

Types of Databases

Hierarchical Databases:

- **Description:** Data is organised in a tree-like structure with parent-child relationships.
- **Example:** IBM Information Management System (IMS)
- **Use Case:** Commonly used in banking and telecommunications for fast access to hierarchical data.

Relational Databases:

- **Description:** Organises data into tables (rows and columns) with predefined relationships, using SQL for querying.
- **Example:** MySQL, PostgreSQL

- **Use Case:** Used for structured data in applications like CRM and ERP systems.

NoSQL Databases:

- **Description:** Designed for unstructured or semi-structured data, offering high scalability and flexibility.
- **Example:** MongoDB, Cassandra, Redis.
- **Use Case:** Best suited for handling big data, real-time web apps, and content management systems.

Network Databases:

- **Description:** Similar to hierarchical databases but with more flexible parent-child relationships, where each child can have multiple parents.
- **Example:** Integrated Data Store (IDS).
- **Use Case:** Used in telecommunications and large systems that require complex relationships between data.

Object-Oriented Databases:

- **Description:** Stores data as objects, integrating database capabilities with object-oriented programming languages.
- **Example:** ObjectDB, db4o.
- **Use Case:** Ideal for complex data structures in software engineering and CAD systems.

Cloud Databases:

- **Description:** Hosted on cloud platforms, providing scalability, accessibility, and reduced infrastructure management.
- **Example:** Amazon RDS, Google Cloud SQL.

- **Use Case:** Commonly used by businesses requiring flexible, scalable data storage with cloud infrastructure.

Centralised Databases:

- **Description:** All data is stored and managed in a single location, typically on a central server.
- **Example:** Centralised database systems in government and large organisations.
- **Use Case:** Best for organisations that need centralised control and management of their data.

Operational Databases:

- **Description:** Real-time databases designed for tracking day-to-day operations and transactions.
- **Example:** Oracle, Microsoft SQL Server.
- **Use Case:** Used in e-commerce, banking, and retail to support day-to-day operations and transactions.

What is a Relational Database?

A relational database organises data into tables (rows and columns) where relationships between data are defined. MySQL and PostgreSQL are examples of relational database systems.

Why Relational Databases?

Relational databases allow easy data retrieval, consistency, and enforcing relationships between tables (like linking a Product_id to its Category).

Example:

Your dataset represents products in an e-commerce platform. Each product has an ID (Product_id), a Brand Name, a Category, and a Discount_Price, among other attributes. This data would be stored in a **table** where each row represents a product and each column is an attribute.

RDBMS vs. DBMS

Database Management Systems (DBMS) and **Relational Database Management Systems (RDBMS)** are both systems used to manage databases, but they differ significantly in structure, functionality, and use cases. Here's a comparison highlighting their key differences:

1. Data Structure:

- **DBMS:** Can manage data in a non-relational format, such as hierarchical or network models. This allows flexibility but may complicate data retrieval.
- **RDBMS:** Organises data into tables with predefined relationships, making it easier to understand and manage.

2. Data Relationships:

- **DBMS:** Often lacks the ability to define relationships between data, which can lead to data inconsistencies.
- **RDBMS:** Defines relationships through keys, ensuring that related data is connected and consistent.

3. Data Integrity and Normalisation:

- **DBMS:** May have limited mechanisms for ensuring data integrity, often relying on application-level checks.

- **RDBMS:** Enforces data integrity and normalisation rules at the database level, which helps maintain accurate and consistent data.

4. Query Language:

- **DBMS:** Might use various query languages, which can limit interoperability and standardisation.
- **RDBMS:** Uses SQL, a standard and widely accepted language for managing relational databases.

Database Languages in DBMS

In Database Management Systems (DBMS), **database languages** are used to interact with the database and perform various operations such as defining, manipulating, querying, and controlling access to the data. These languages are broadly classified into four types: **Data Definition Language (DDL)**, **Data Manipulation Language (DML)**, **Data Query Language (DQL)**, and **Data Control Language (DCL)**.

Data Definition Language (DDL)

Data Definition Language (DDL) is a subset of SQL used to **define** and **manage the structure** of a database. DDL commands allow you to create, modify, and remove database objects like tables, indexes, and schemas.

Common DDL Commands:

CREATE: Used to **create new objects** in the database such as tables, indexes, and views.

ALTER: Used to **modify an existing object**, such as adding or removing columns from a table.

DROP: Used to **delete** an existing database object, like a table or an index.

TRUNCATE: Used to **remove all rows** from a table without deleting the table itself. It is faster than DELETE but doesn't allow rollback.

Why is Data Definition Language (DDL) important?

1. **Establishing the Initial Structure:** DDL is used to create the foundational elements of a database, such as tables, schemas, and indexes. This initial setup defines how data will be organised, stored, and accessed, setting the stage for all subsequent operations. A well-designed structure is crucial for efficient data management and retrieval.
2. **Modifying and Optimising the Structure:** As organisations evolve and their data needs change, DDL commands (like ALTER) allow for modifications to the database schema. This may include adding new fields, altering existing ones, or changing data types. Such flexibility is vital for optimising database performance and ensuring that the structure aligns with the organisation's current requirements.
3. **Maintaining Organization and Efficiency:** DDL enables the deletion of unnecessary or obsolete objects (using DROP commands) from the database. This helps keep the database organised, reduces clutter, and enhances efficiency by minimising the amount of data the system needs to manage. A streamlined database structure contributes to faster query performance and easier maintenance.
4. **Supporting Data Integrity and Constraints:** DDL also allows for the definition of constraints (e.g., primary keys, foreign keys,

and unique constraints) that help enforce data integrity and ensure that relationships between data elements are properly maintained. This is crucial for preventing data anomalies and maintaining the accuracy and reliability of the information stored in the database.

Data Manipulation Language (DML)

DML (Data Manipulation Language) is used to manage and manipulate the data stored in database objects created using Data Definition Language (DDL). DML allows you to insert, update, delete, and retrieve records from tables within the database.

Common DML Commands:

INSERT: Adds new records into a table.

UPDATE: Modifies existing records in a table.

DELETE: Removes records from a table.

MERGE: Combines INSERT, UPDATE, and DELETE operations into a single statement. Typically used for synchronising data between two datasets.

Why is Data Manipulation Language (DML) important?

1. **Inserting Data:** DML allows for the addition of new records to the database, such as new products, customers, or orders.

Keeping the catalog and user database current ensures that customers have access to the latest offerings and services.

2. **Updating Data:** DML facilitates the modification of existing records. This includes adjusting product prices, updating stock levels, and altering customer information to reflect changes such as discounts, promotions, or restocks. Timely updates help maintain accurate and relevant information for both the business and its customers.
3. **Deleting Data:** DML provides the capability to remove outdated or unnecessary records, such as discontinued products or inactive customer accounts. Maintaining a clean database enhances performance and relevance, making it easier for customers to find what they need.
4. **Merging Data:** DML enables the synchronisation of external datasets with the existing database. For instance, integrating updated information from suppliers or partner systems ensures that product details remain accurate and up-to-date, improving the overall customer experience.

Data Query Language (DQL)

Data Query Language (DQL) is a subset of Data Manipulation Language (DML) and is specifically used to query and retrieve data from a database. The core command in DQL is SELECT, which allows users to fetch data from one or more tables.

Common DQL Commands

SELECT: Retrieves data from one or more tables.

WHERE Clause: Adds a condition to filter the rows returned by a SELECT query.

GROUP BY Clause: Groups rows that share the same value in one or more columns and applies aggregate functions like COUNT(), AVG(), SUM()).

Why is Data Query Language (DQL) Important?

1. **Data Retrieval:** DQL is primarily used for retrieving specific data from databases, making it essential for various tasks such as data analysis, reporting, and operational decision-making. By executing SELECT statements, users can access exactly the information they need without having to sift through vast amounts of data. This capability is crucial for timely and informed decision-making in business environments.
2. **Data Filtering:** DQL allows users to apply conditions (using the WHERE clause) to filter the data being retrieved. This means users can specify precise criteria for the data they want, enabling them to extract only the relevant records. For instance, a sales manager might want to see only transactions above a certain amount, which can help them focus on high-value customers or sales trends.

3. **Data Grouping and Aggregation:** With commands like GROUP BY and aggregate functions (such as SUM, AVG, COUNT), DQL enables users to summarise and analyse data efficiently. For example, businesses can calculate total sales per product or average customer spending, providing valuable insights into performance and trends. This capability is essential for generating reports and making data-driven strategic decisions.
4. **Data Sorting:** The ORDER BY clause in DQL allows users to organise and sort query results based on specific columns, improving the clarity and readability of the retrieved information. For example, a list of products can be sorted by price or name, making it easier for users to find what they need. This organisation enhances user experience and helps in data analysis.
5. **Facilitating Complex Queries:** DQL supports the use of joins and subqueries, enabling users to retrieve data from multiple tables in a single query. This ability to perform complex queries helps in understanding relationships between different data entities, providing a comprehensive view of the data landscape.

Data Control Language (DCL)

Data Control Language (DCL) is a subset of SQL used to control access to data within a database. It allows database administrators to manage user permissions and security by granting or revoking privileges on database objects. This is majorly done by DB Admins and not by DS professionals

Common DCL Commands

GRANT Command: Grants specific permissions to users or roles, allowing them to perform certain actions on database objects (like tables, views, etc.).

REVOKE Command: Removes specific permissions from users or roles, preventing them from performing certain actions on database objects.

Transaction Control Language (TCL)

TCL (Transaction Control Language) is used to manage transactions in a database. A transaction is a sequence of operations that is treated as a single logical unit, where either all the changes are applied to the database, or none at all. TCL helps ensure that the database remains in a consistent state, even in the case of system failures or errors. This is majorly done by DB Admins and not by DS professionals

Common TCL Commands

COMMIT: Permanently saves all the changes made during the current transaction.

ROLLBACK: Reverts all the changes made during the current transaction, undoing DML operations since the last COMMIT or SAVEPOINT.

SAVEPOINT: Creates a point in the transaction to which you can roll back without affecting the entire transaction.

Introduction to SQL

What is SQL?

SQL (Structured Query Language) is a standardised language used to manage and manipulate relational databases. SQL allows you to perform various operations like retrieving, inserting, updating, and deleting data in a database. It also helps in managing database structures such as tables, views, indexes, and controlling user access.

What is MySQL?

MySQL is an open-source relational database management system (RDBMS) that uses SQL to interact with the database. It is one of the most popular databases in the world and is commonly used for web applications, data storage, and backend services. MySQL allows for efficient and scalable management of large datasets.

Why Use MySQL?

MySQL is a widely used open-source relational database management system (RDBMS) known for its speed, reliability, and flexibility. It is particularly suitable for managing structured data, making it a popular choice for web applications and data-driven projects.

Install MySQL

[Link](#) for Windows

MySQL Workbench:

MySQL Workbench is a powerful tool that provides a graphical interface for designing, managing, and running queries on MySQL databases. It simplifies database administration and development tasks.

Features:

- **Visual Database Design:** Users can create and modify database schemas visually using Entity-Relationship (ER) diagrams.
- **SQL Development:** Allows writing, executing, and debugging SQL queries in a user-friendly interface.
- **Database Administration:** Offers tools for user management, backup, and performance tuning.
- **Data Migration:** Supports migrating databases from other systems to MySQL.
- **Server Administration:** Provides a visual dashboard for monitoring server performance and status.

Usage:

- Ideal for users who prefer a visual interface and need to manage databases without extensive command-line knowledge.
- Commonly used by database administrators, developers, and data analysts for daily database tasks.

MySQL Shell

MySQL Shell is a command-line interface for interacting with MySQL databases. It supports both SQL and NoSQL (document) operations, allowing for flexibility in database management.

Features:

- **Multiple Modes:** Supports SQL mode, JavaScript mode, and Python mode for various scripting needs.
- **NoSQL Support:** Allows working with MySQL Document Store for handling JSON documents.
- **Scripting:** Facilitates automation of tasks using scripts in JavaScript or Python.
- **Advanced Features:** Provides advanced capabilities for database management, such as executing batch operations and handling transactions.

Usage:

- Ideal for users who are comfortable with command-line interfaces and need to perform complex operations or automation.
- Commonly used by developers, data engineers, and system administrators for scripting and advanced database operations.

Basic Database Operations

Here's a quick overview of essential SQL commands you'll use frequently:

Create a Database:

Unset

```
CREATE DATABASE my_database;
```

Use a Database:

Unset

```
USE my_database;
```

Basic SQL Operations

CREATE: Used to create a new table in the database.

Unset

```
CREATE TABLE Products (  
    Product_id INT PRIMARY KEY,  
    Brand_Name VARCHAR(100),  
    Category VARCHAR(50),  
    DiscountPrice DECIMAL(10, 2)  
);
```

INSERT: Used to add new data into a table.

Unset

```
INSERT INTO Products (Product_id, Brand_Name, Category,  
DiscountPrice)  
VALUES (1, 'BrandA', 'Category1', 19.99);
```

SELECT: Retrieves data from one or more tables.

Unset

```
SELECT * FROM Products;
```

UPDATE: Modifies existing data within a table.

Unset

```
UPDATE Products SET DiscountPrice = 15.99 WHERE Product_id = 1;
```

DELETE: Removes records from a table.

Unset

```
DELETE FROM Products WHERE Product_id = 1;
```

Example:

Creating a Table

Unset

```
CREATE TABLE Products (  
    Product_id INT PRIMARY KEY,  
    URL VARCHAR(255),  
    Brand_Name VARCHAR(100),  
    Category VARCHAR(50),  
    Individual_category VARCHAR(50),  
    Category_by_Gender VARCHAR(20),  
    Description TEXT,  
    DiscountPrice DECIMAL(10, 2),  
    OriginalPrice DECIMAL(10, 2),  
    DiscountOffer VARCHAR(10),  
    SizeOption VARCHAR(50),  
    Ratings DECIMAL(3, 2),  
    Reviews INT  
);
```

This SQL query is creating a table called Products with various columns to store product-related data in a structured format. Let's break down each part of the query for better understanding:

1. **CREATE TABLE** Products:

This statement creates a new table named Products in the database. The structure of the table is defined in the parentheses, specifying the column names and data types.

2. **Column Definitions:**

The columns in the Products table represent the various attributes of each product. Here's what each column represents:

- **Product_id INT PRIMARY KEY:**
 - Product_id: This is an integer (INT) column that stores the unique identifier for each product.
 - PRIMARY KEY: The Product_id is set as the primary key, meaning each product must have a unique Product_id, and this field cannot be NULL.
- **URL VARCHAR(255):**
 - URL: A VARCHAR column (variable-length character string) that can hold up to 255 characters.
 - This stores the URL of the product page or image.
- **Brand_Name VARCHAR(100):**
 - Brand_Name: A VARCHAR column that stores the name of the product's brand, with a maximum length of 100 characters.
- **Category VARCHAR(50):**
 - Category: A VARCHAR column that stores the general category of the product, such as "Electronics" or "Clothing", with a maximum length of 50 characters.

- **Individual_category VARCHAR(50):**
 - Individual_category: A VARCHAR column that stores a more specific subcategory of the product. For instance, "Laptops" could be a subcategory under "Electronics."
- **Category_by_Gender VARCHAR(20):**
 - Category_by_Gender: A VARCHAR column that stores gender-specific categories if applicable, such as "Men", "Women", or "Unisex". This can hold up to 20 characters.
- **Description TEXT:**
 - Description: A TEXT column that stores a more detailed description of the product. The TEXT data type can hold large blocks of text, suitable for product descriptions.
- **DiscountPrice DECIMAL(10, 2):**
 - DiscountPrice: A DECIMAL column that stores the discounted price of the product. The format DECIMAL(10, 2) allows for up to 10 total digits, with 2 digits after the decimal point (e.g., 99999999.99).
- **OriginalPrice DECIMAL(10, 2):**
 - OriginalPrice: Another DECIMAL column that stores the original price of the product, also with up to 10 digits in total and 2 digits after the decimal.
- **DiscountOffer VARCHAR(10):**
 - DiscountOffer: A VARCHAR column that stores information about the discount offer (e.g., "10% off"), with a maximum length of 10 characters.

- **SizeOption VARCHAR(50):**
 - SizeOption: A VARCHAR column that stores the available size options for the product (e.g., "S, M, L, XL"), with a maximum length of 50 characters.
- **Ratings DECIMAL(3, 2):**
 - Ratings: A DECIMAL column that stores the rating of the product, with up to 3 total digits and 2 digits after the decimal point (e.g., 4.50 out of 5).
- **Reviews INT:**
 - Reviews: An INT column that stores the number of customer reviews for the product.

Database Design Basics

What is Database Design?

Database design is the process of structuring and organising data into tables, relationships, and constraints to ensure data is stored efficiently and logically. It focuses on avoiding redundancy, ensuring consistency, and allowing for easy data retrieval.

Why is Database Design Important?

Proper database design is crucial because it:

- **Reduces Redundancy:** Eliminates repetitive data, which saves storage space and simplifies updates.
- **Ensures Data Integrity:** Guarantees accuracy and consistency through constraints like primary and foreign keys.

- **Improves Query Performance:** Speeds up data retrieval and manipulation by optimising how data is stored and related.

Main Concepts in Database Design

Tables

The structure that stores data in rows and columns.

- In an E-commerce dataset, you could have multiple tables to store different types of data, such as:
 - **Products** table: Stores product information like Product_id, Brand_Name, Category, DiscountPrice, etc.
 - **Categories** table: Stores category details, such as Category_id and Category_name.
 - **Customers** table: Stores customer data like Customer_id, Name, Email, etc.

Example Products table:

Product_id	Brand_Name	Category_id	DiscountPrice	OriginalPrice
1	Nike	2	50.00	75.00
2	Adidas	3	40.00	60.00

Primary Key

The Primary Key ensures each record is unique within the table. In your **Products** table, Product_id is the primary key that uniquely identifies each product.

Example: In the **Categories** table, Category_id would serve as the primary key to uniquely identify each category:

Category_id	Category_name
1	Clothing

2	Footwear
---	----------

Foreign Key

A **Foreign Key** links related records between tables. In the **Products** table, `Category_id` acts as a foreign key that references the `Category_id` in the **Categories** table.

Example: We have two tables: **Products** and **Categories**. The `Category_id` column in the **Products** table is a **Foreign Key** that references the `Category_id` in the **Categories** table. This relationship ensures that every product must belong to a valid category, maintaining data integrity.

KEY NOTE : *DDL and DML commands are mainly one time activities and as a Data Science professional, we are more focused towards DQL (Select) queries.*

Categories Table (Referenced Table)

Category_id	Category_name	Category_Description
1	Footwear	Shoes, sandals, sneakers
2	Clothing	Shirts, pants, jackets
3	Accessories	Belts, hats, gloves

Products Table (Table with Foreign Key)

Product_id	Product_name	Brand_Name	Category_id	DiscountPrice	OriginalPrice
1	Running Shoes	Nike	1	50.00	75.00
2	T-Shirt	Adidas	2	20.00	30.00
3	Leather Belt	Gucci	3	40.00	60.00

Relationships

What are Relationships in Database Design?

Relationships in database design define how data in different tables is connected or related to each other. They help maintain the integrity and organisation of data within a relational database. The main types of relationships are:

1. **One-to-One** : Each record in one table corresponds to exactly one record in another table.
2. **One-to-Many** : A record in one table can relate to multiple records in another table, but each record in the second table relates to only one record in the first table.
3. **Many-to-Many** : Records in one table can relate to multiple records in another table, and vice versa. This usually requires a junction table.

Why are Relationships Important?

1. **Data Integrity**: Relationships help ensure data consistency and accuracy. For example, foreign keys enforce rules that prevent orphan records (records that reference non-existent entries in related tables).
2. **Reduction of Redundancy**: By organising data into related tables, redundancy is minimised. For instance, category details are stored in a single Categories table rather than duplicated in every product entry.
3. **Efficient Data Retrieval**: Relationships allow for efficient querying across multiple tables, making it easier to retrieve

related data. For instance, you can quickly find all products under a specific category.

4. **Maintenance and Scalability:** Properly defined relationships make databases easier to maintain and scale. Changes to data only need to occur in one place, reducing the risk of errors.

Inserting Sample Data:

Unset

```
--Insert sample employees
```

```
INSERT INTO employees (employee_id, employee_name, manager_id)
VALUES (1, 'Manager A', NULL),      --Manager A has no manager
       (2, 'Employee B', 1),        --Employee B reports to Manager A
       (3, 'Employee C', 1),        --Employee C reports to Manager A
       (4, 'Employee D', 2);        --Employee D reports to Employee B
```

- The employees table includes a manager_id column that serves as a foreign key referencing the employee_id within the same table.
- This structure allows for establishing a hierarchy where each employee can have one manager, while a manager can oversee multiple employees. For example, Manager A supervises both Employee B and Employee C, while Employee D reports to Employee B.

Normalization

What is Normalization?

Normalization is a systematic process in database design that helps organise data by dividing large tables into smaller, more manageable

ones. It reduces redundancy and dependency by ensuring that each piece of data is stored in only one place.

The goal of normalization is to break down a database into tables and define relationships between them while avoiding data anomalies such as duplication, insertion, update, or deletion anomalies. This is achieved through different levels of normalization, known as **normal forms (NF)**, including 1NF, 2NF, 3NF, and beyond.

Why Do We Need Normalization?

Normalization is an essential process in relational database design for organising data to reduce redundancy and improve data integrity. It provides several critical benefits that make databases efficient and reliable for both storage and querying. Let's explore the key reasons we need normalization:

- Minimises Data Redundancy
- Improves Data Integrity and Consistency
- Avoids Update, Insert, and Delete Anomalies
- Optimizes Query Performance
- Easier Maintenance and Scalability
- Clearer Database Structure

Normal Forms (NF)

Normal Forms (NF) are a set of rules used to ensure that a relational database is structured efficiently, minimising redundancy and dependency while enhancing data integrity. They are part of the database normalization process and consist of a hierarchy of rules,

from **First Normal Form (1NF)** to more advanced forms like **Boyce-Codd Normal Form (BCNF)** and beyond.

Types of Keys in Database

In database design, **keys** are essential for ensuring the accuracy, integrity, and efficiency of data storage and retrieval. They help uniquely identify records, establish relationships between tables, and enforce constraints to maintain data integrity. Below are the different types of keys used in databases:

1. Primary Key (PK)

A **primary key** is a unique identifier for a record in a table. It ensures that no two rows have the same value for the primary key and that the primary key value is never null.

Example: In an e-commerce database

Unset

```
CREATE TABLE Customers (  
    Customer_ID INT PRIMARY KEY,  
    Customer_Name VARCHAR(100),  
    Email VARCHAR(100)  
);
```

Customer_ID is the primary key, ensuring that each customer has a unique identifier.

2. Composite Key

A **composite key** is a combination of two or more columns that together uniquely identify a record. None of the individual columns can uniquely identify the record on their own.

Example: In a **ProductOrders** table that records the orders of each product.

```
Unset
CREATE TABLE ProductOrders (
    Order_ID INT,
    Product_ID INT,
    Quantity INT,
    PRIMARY KEY (Order_ID, Product_ID)
);
```

The combination of Order_ID and Product_ID acts as the primary key. A single Order_ID may not uniquely identify a record, but the combination of both Order_ID and Product_ID will.

3. Candidate Key

A **candidate key** is a column or set of columns that can uniquely identify a record. A table can have multiple candidate keys, but one of them is chosen as the primary key.

Example: In a **Products** table

```
Unset
CREATE TABLE Products (
    Product_ID INT,
    Product_Code VARCHAR(50),
    Product_Name VARCHAR(100),
    PRIMARY KEY (Product_ID)
);
```

Both Product_ID and Product_Code could uniquely identify a product. They are both candidate keys, but only one is chosen as the primary key (Product_ID).

4. Alternate Key

An **alternate key** is any candidate key that is not chosen as the primary key.

Example: From the above **Products** table Product_Code is an alternate key because it is a candidate key that wasn't chosen as the primary key.

5. Foreign Key (FK)

A **foreign key** is a column or set of columns that establishes a relationship between two tables. It refers to the primary key in another table, ensuring referential integrity.

Example: In a **ProductOrders** table, Customer_ID might refer to the Customers table.

Unset

```
CREATE TABLE ProductOrders (  
    Order_ID INT PRIMARY KEY,  
    Product_ID INT,  
    Customer_ID INT,  
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)  
);
```

Customer_ID is a foreign key, establishing a link between the ProductOrders table and the Customers table.

THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)